Library Management System

Spring Boot JSP CRUD Project Submission

Saswata Das

2024eb02309

1. Project Overview

This project is a Spring Boot MVC application for managing a small library dataset. The two selected entities are `Author` and `Book`. The application supports create, read, and update operations for both entities using JSP pages, Spring MVC controllers, service classes, Spring Data JPA repositories, and an H2 database.

The application also includes sample data population, database constraints, custom inner join query support, exception handling for duplicate values, styled JSP views, and unit tests for repository and service logic.

Technology Stack

Layer	Technology
Backend	Spring Boot 3.3.5
Web MVC	Spring MVC
Views	JSP, JSTL, Expression Language
Persistence	Spring Data JPA, Hibernate
Database	H2 in-memory database
Validation	Jakarta Bean Validation
Testing	JUnit 5, Mockito, Spring Boot Test
Build Tool	Maven

2. Entity Relationship Design

The system has a one-to-many relationship between authors and books.

Entity	Main Fields	Constraints
`Author`	`id`, `name`, `email`, `country`	`name` unique, `email` unique, required fields
`Book`	`id`, `title`, `isbn`, `genre`, `publishedYear`, `author`	`isbn` unique, author required

Relationship:

```
Author 1 ---- * Book
```

One author can write many books. Each book belongs to one author. In the database, the `books` table contains `author_id` as a foreign key referencing the `authors` table.

Entity Code

```
@Entity
@Table(name = "authors")
public class Author {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 100)
    private String name;

    @Column(nullable = false, unique = true, length = 120)
    private String email;

    @Column(nullable = false, length = 80)
    private String country;

    @OneToMany(mappedBy = "author", cascade = CascadeType.ALL,
            orphanRemoval = true, fetch = FetchType.LAZY)
    private List<Book> books = new ArrayList<>();
}

@Entity
```

```
@Table(name = "books")
public class Book {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 150)
    private String title;

    @Column(nullable = false, unique = true, length = 20)
    private String isbn;

    @Column(nullable = false, length = 80)
    private String genre;

    @Column(nullable = false)
    private Integer publishedYear;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "author_id", nullable = false)
    private Author author;
}
```

3. Database Population

The tables are generated automatically using JPA and Hibernate. `DataSeeder` runs on application startup and inserts 10 authors and 10 books.

```
@Component
public class DataSeeder implements CommandLineRunner {
    @Override
    public void run(String... args) {
        if (authorRepository.count() > 0) {
            return;
        }

        List<Author> authors = authorRepository.saveAll(List.of(
                new Author("Aarav Mehta", "aarav.mehta@example.com", "India"),
                new Author("Maya Iyer", "maya.iyer@example.com", "India"),
                new Author("John Carter", "john.carter@example.com", "USA")
                // 10 authors total
        ));

        bookRepository.saveAll(List.of(
                new Book("Spring in Action", "9781617294945", "Technology", 2022, authors.get(0)),
                new Book("Clean Code Notes", "9780132350884", "Programming", 2020, authors.get(1)),
                new Book("Cloud Patterns", "9781492050285", "Technology", 2021, authors.get(2))
                // 10 books total
        ));
    }
}
```

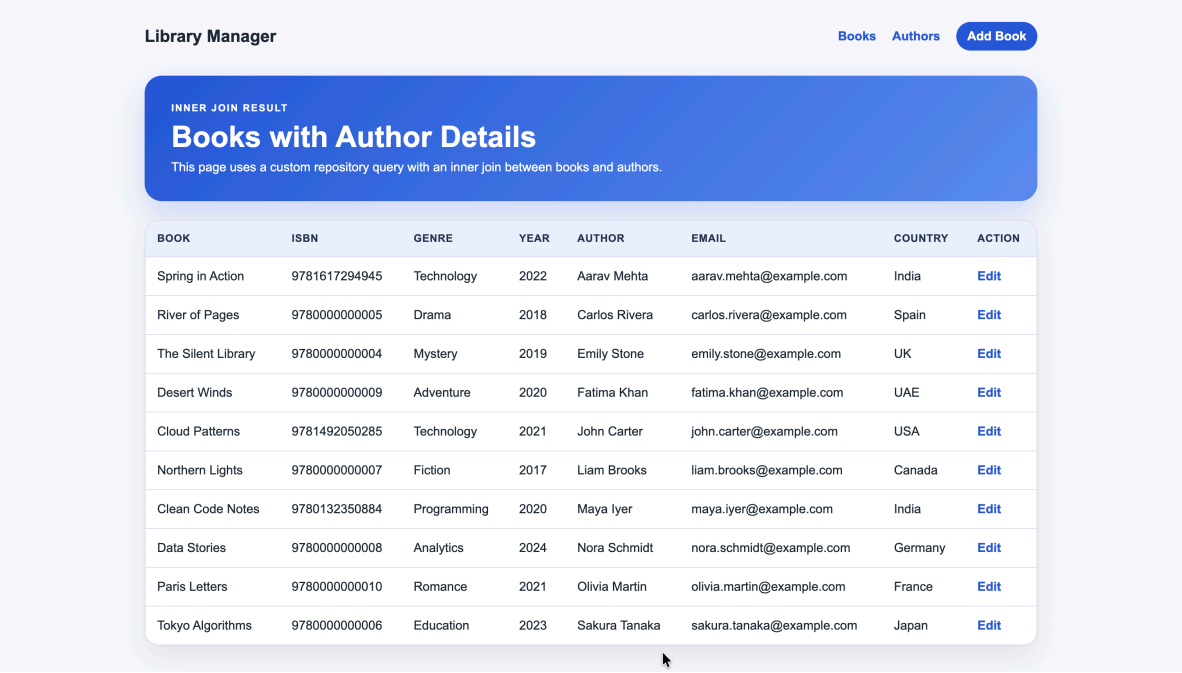

Figure 1: Books list with joined author details.

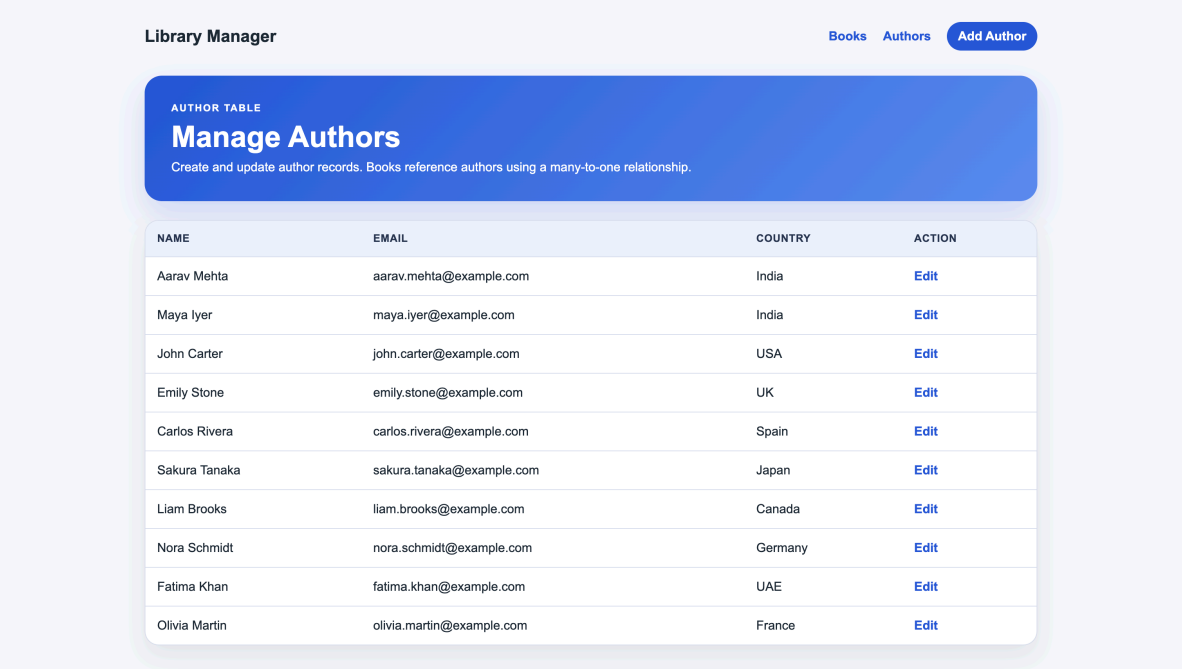

Figure 2: Authors table with seeded and created records.

4. Repository Layer

Both repositories extend `JpaRepository`. The book repository includes a custom inner join query that returns book and author data as a DTO projection.

```
public interface AuthorRepository extends JpaRepository<Author, Long> {
    boolean existsByEmail(String email);
    boolean existsByName(String name);
}

public interface BookRepository extends JpaRepository<Book, Long> {
    boolean existsByIsbn(String isbn);

    @Query("""
            select new com.example.librarymanagement.dto.BookAuthorView(
                b.id, b.title, b.isbn, b.genre, b.publishedYear,
                a.id, a.name, a.email, a.country
            )
            from Book b
```

```
            inner join b.author a
            order by a.name, b.title
            """)
    List<BookAuthorView> findBooksWithAuthors();
}
```

5. Service Layer

Service classes contain the business logic and connect controllers to repositories. The service layer also checks integrity conditions such as duplicate ISBN and missing author selection.

```
public Book create(Book book) {
    if (bookRepository.existsByIsbn(book.getIsbn())) {
        throw new DataIntegrityViolationException("A book with this ISBN already exists.");
    }

    Long authorId = book.getAuthor() == null ? null : book.getAuthor().getId();
    if (authorId == null) {
        throw new DataIntegrityViolationException("Please select an author for the book.");
    }

    Author author = authorService.findById(authorId);
    book.setAuthor(author);
    return bookRepository.save(book);
}

public Book update(Long id, Book bookDetails) {
    Book book = findById(id);
    Author author = authorService.findById(bookDetails.getAuthor().getId());

    book.setTitle(bookDetails.getTitle());
    book.setIsbn(bookDetails.getIsbn());
    book.setGenre(bookDetails.getGenre());
    book.setPublishedYear(bookDetails.getPublishedYear());
    book.setAuthor(author);

    return bookRepository.save(book);
}
```

6. Controller Layer

Spring MVC controllers handle request mapping, form binding, validation errors, success messages, and redirects.

```
@GetMapping
public String listBooks(Model model) {
    model.addAttribute("bookAuthorRows", bookService.findBooksWithAuthors());
    return "books/list";
}

@PostMapping
public String createBook(@Valid @ModelAttribute("book") Book book,
                         BindingResult bindingResult,
                         Model model,
                         RedirectAttributes redirectAttributes) {
    if (bindingResult.hasErrors()) {
        prepareBookForm(model, "Add Book", "/books");
        return "books/form";
    }

    try {
        bookService.create(book);
        redirectAttributes.addFlashAttribute("successMessage", "Book added successfully.");
        return "redirect:/books";
    } catch (DataIntegrityViolationException ex) {
        bindingResult.reject("integrity", ex.getMostSpecificCause().getMessage());
```

```
        prepareBookForm(model, "Add Book", "/books");
        return "books/form";
    }
}
```

7. View Layer

The UI is implemented using JSP pages with JSTL and Spring form tags. CSS is used for layout, tables, forms, buttons, alerts, and page structure.

Main JSP pages:

Page	Purpose
`books/list.jsp`	Displays books with author details using the inner join result
`books/form.jsp`	Adds and updates books
`authors/list.jsp`	Displays authors
`authors/form.jsp`	Adds and updates authors
`error.jsp`	Displays handled errors

8. Create Operation

Add Author

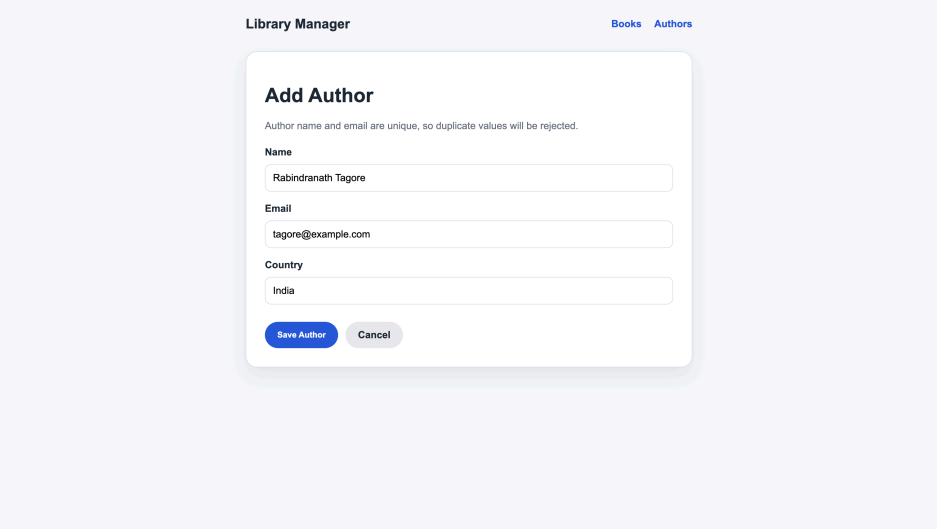

Figure 3: Add author form.

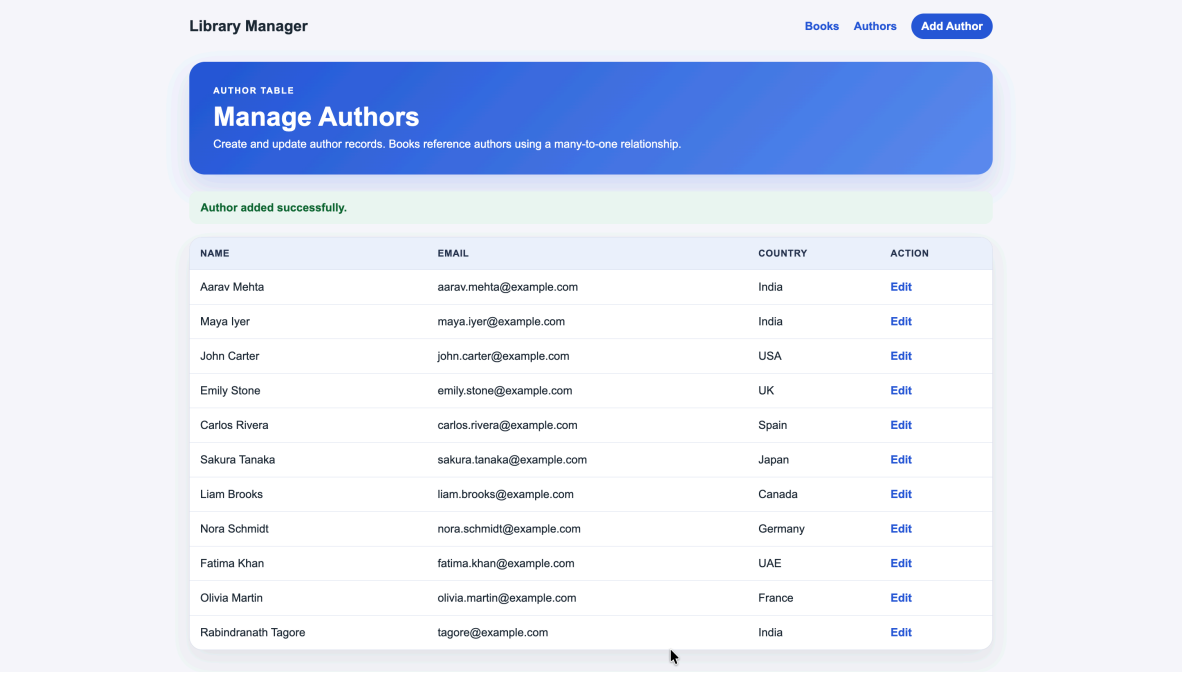

Figure 4: Author added successfully.

Add Book

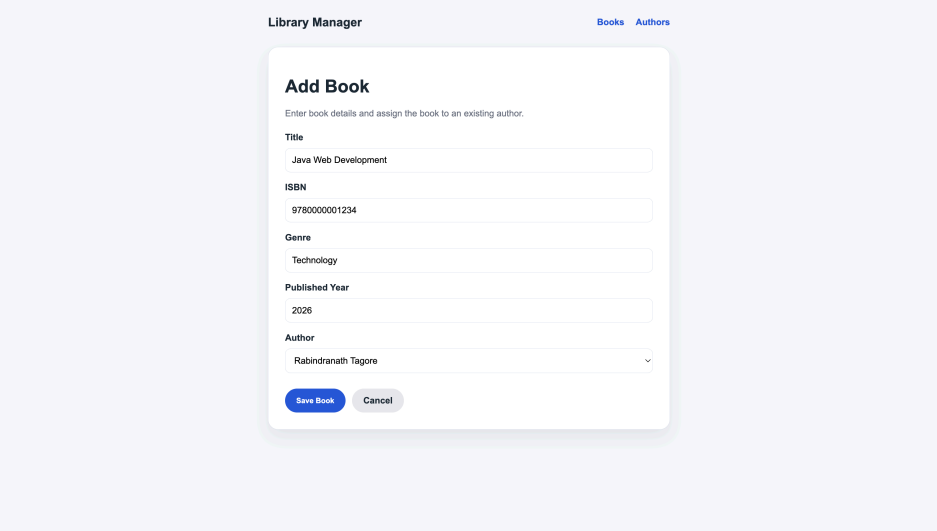

Figure 5: Add book form with author selection.

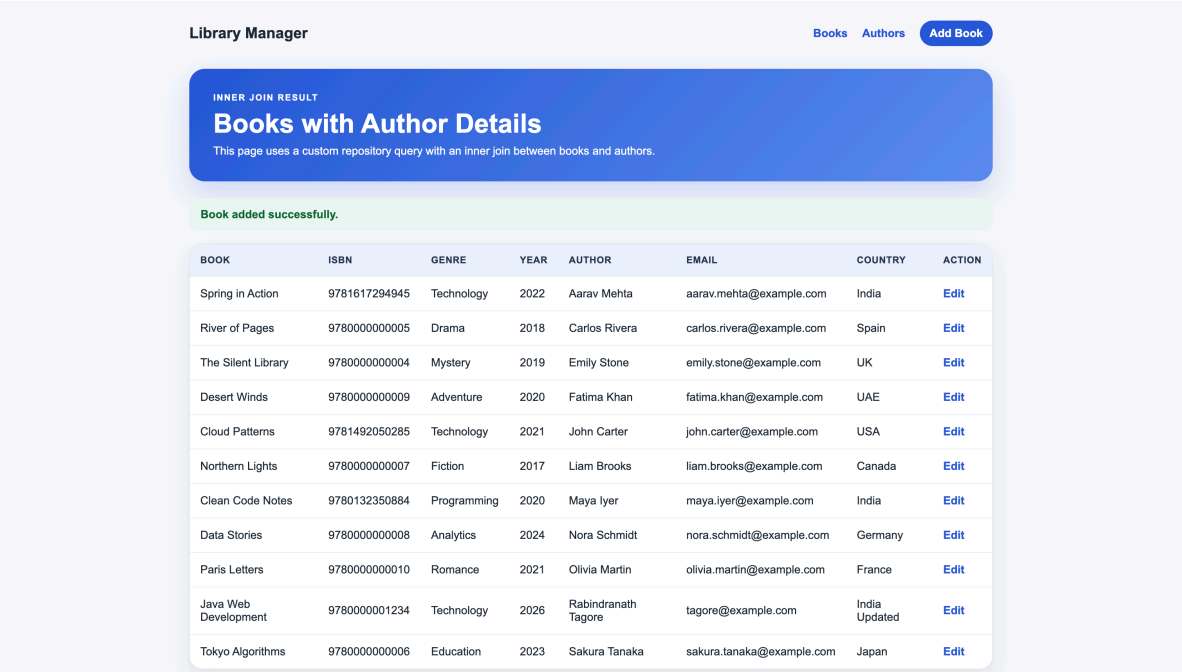

Figure 6: Book added successfully.

9. Read Operation

The read operation fetches data from the service layer and binds it to JSP views through the controller model. The `/books` page displays joined book-author data. The `/authors` page displays all authors.

```
@GetMapping
public String listBooks(Model model) {
    model.addAttribute("bookAuthorRows", bookService.findBooksWithAuthors());
    return "books/list";
}
```

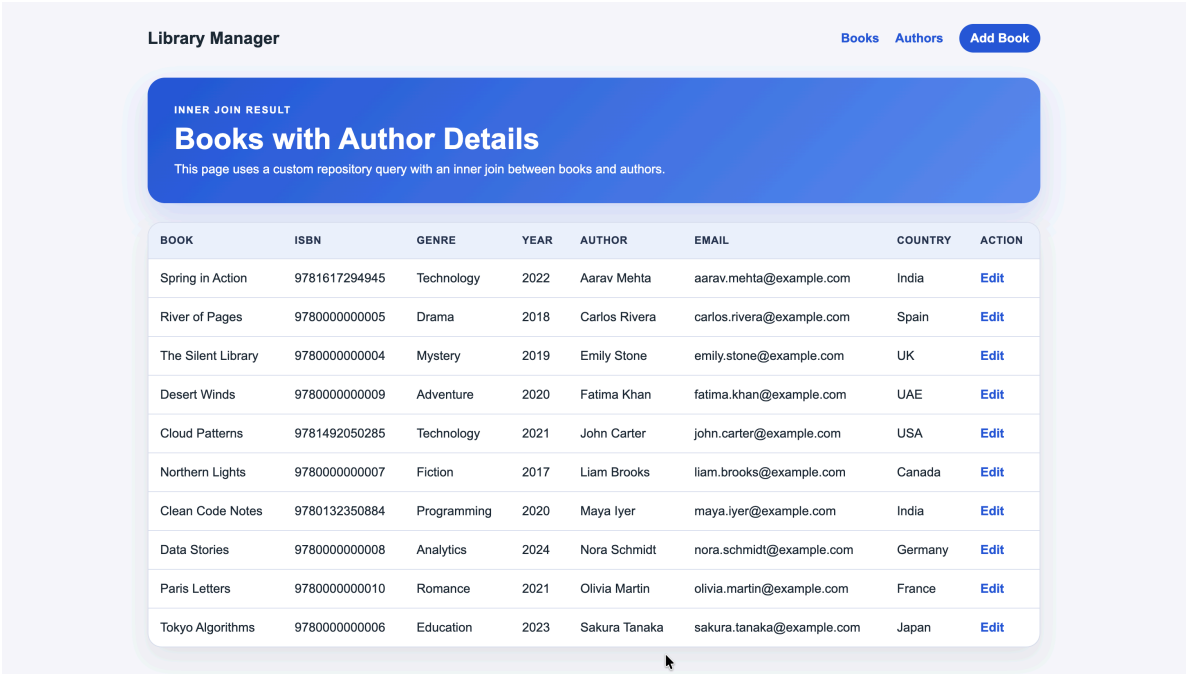

Figure 7: Books read page.

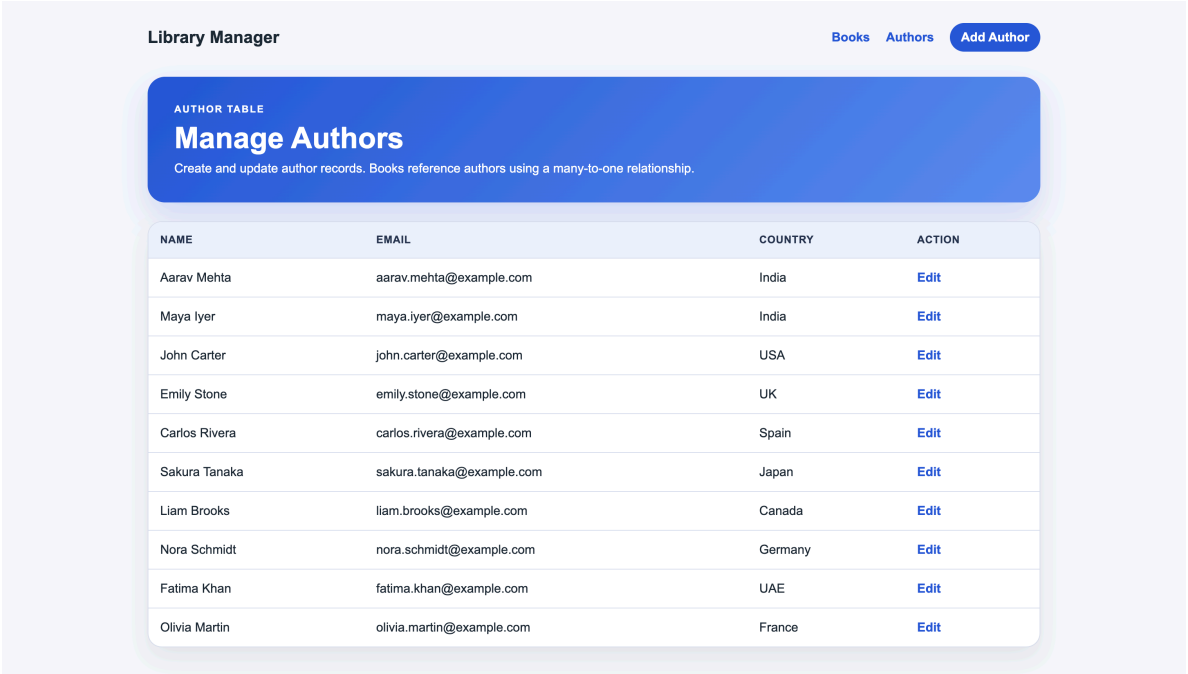

Figure 8: Authors read page.

10. Update Operation

Update Author

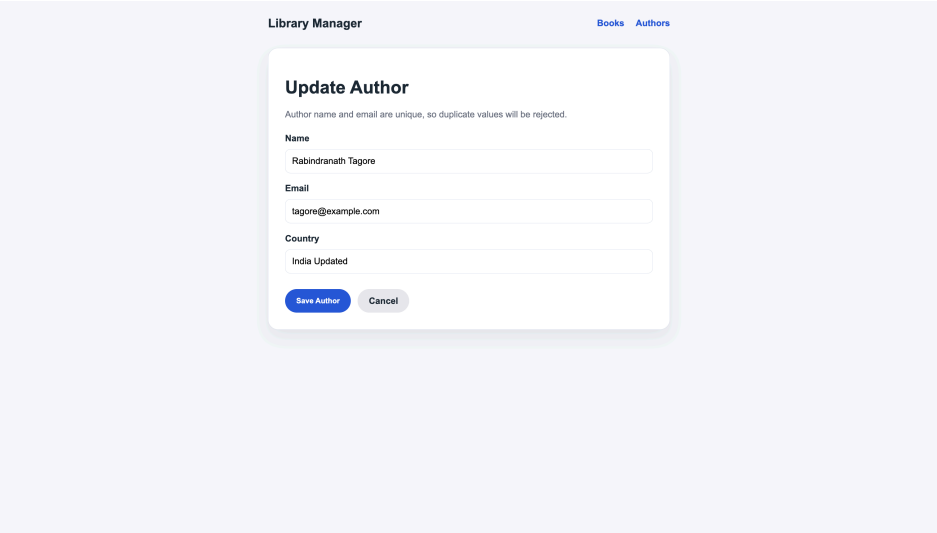

Figure 9: Edit author form.

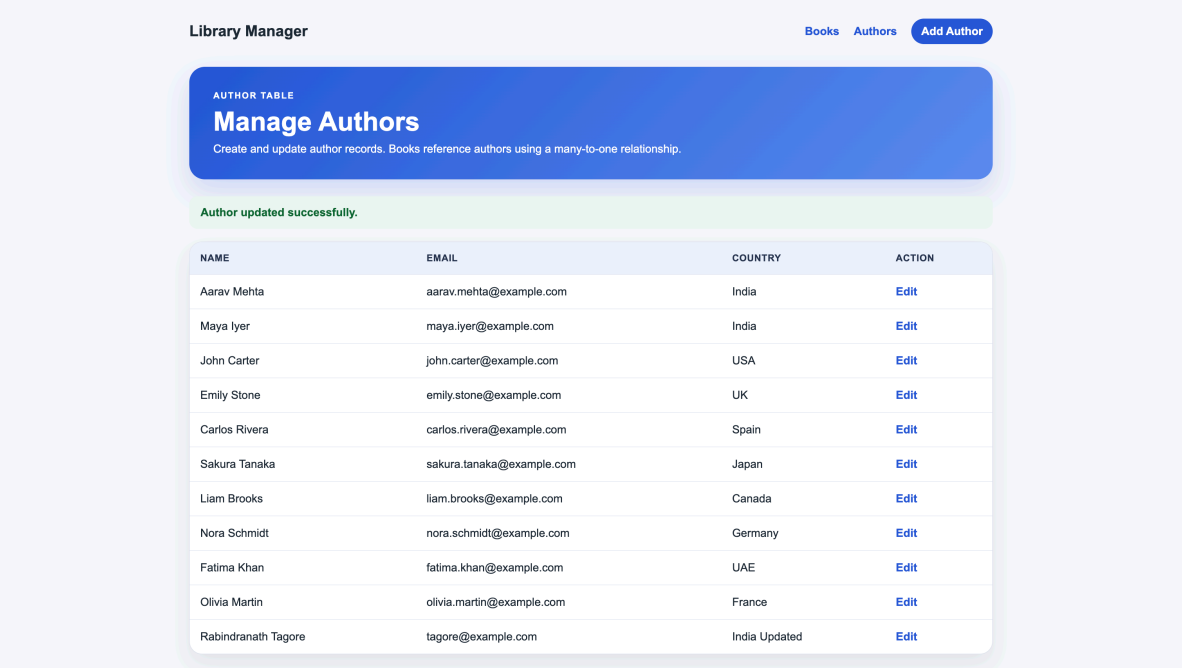

Figure 10: Author updated successfully.

Update Book

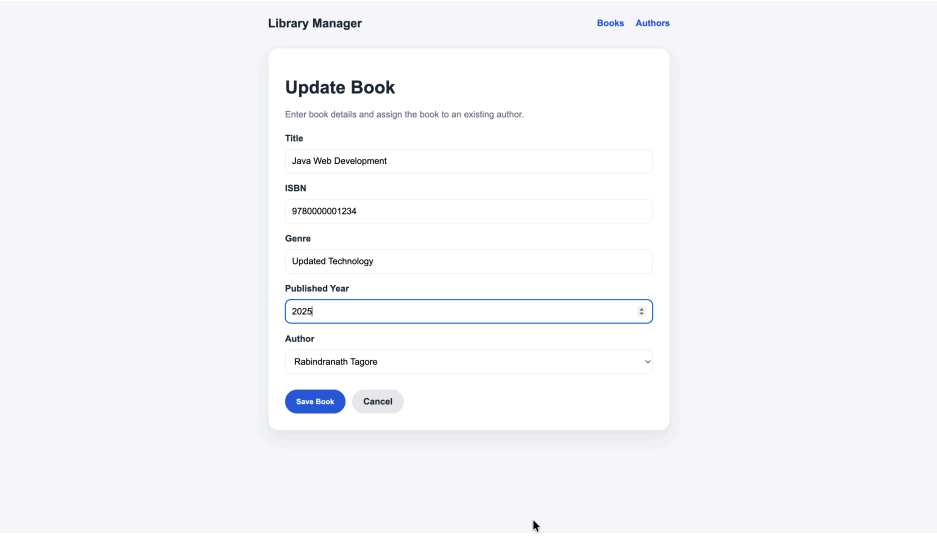

Figure 11: Edit book form.

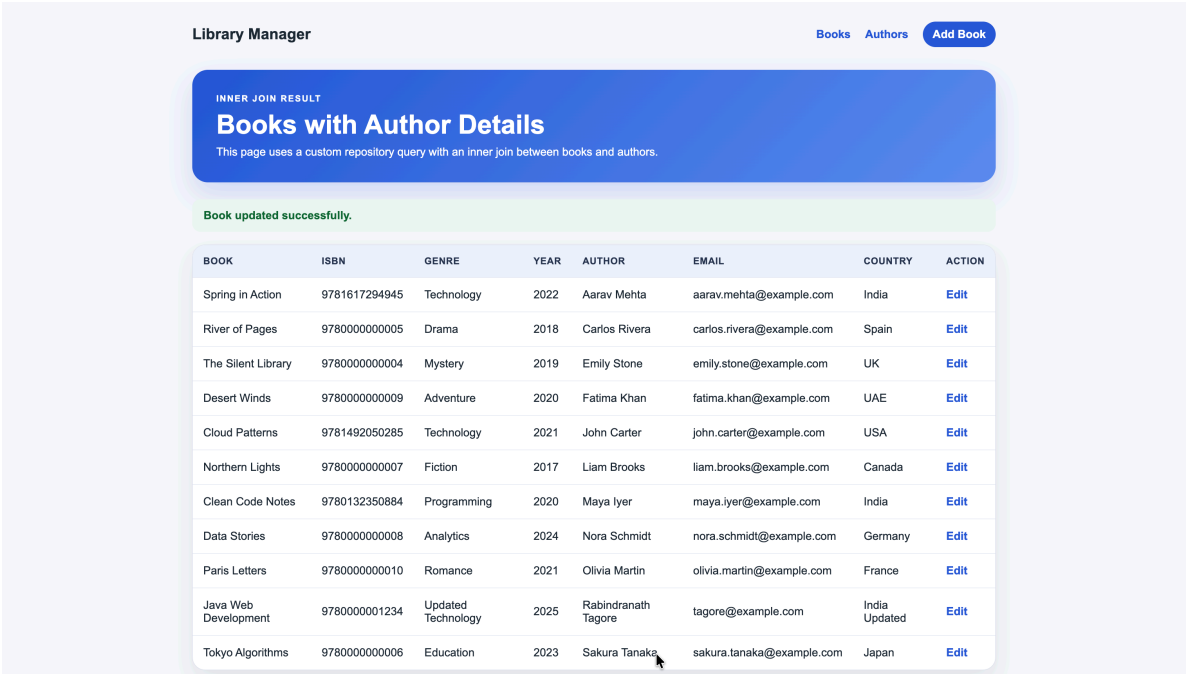

Figure 12: Book updated successfully.

11. Exception Handling

Integrity violations are handled for duplicate author name/email and duplicate book ISBN. The form is shown again with an error message instead of crashing the application.

```
try {
    authorService.create(author);
    redirectAttributes.addFlashAttribute("successMessage", "Author added successfully.");
    return "redirect:/authors";
} catch (DataIntegrityViolationException ex) {
    bindingResult.reject("duplicate", ex.getMostSpecificCause().getMessage());
    return "authors/form";
}
```

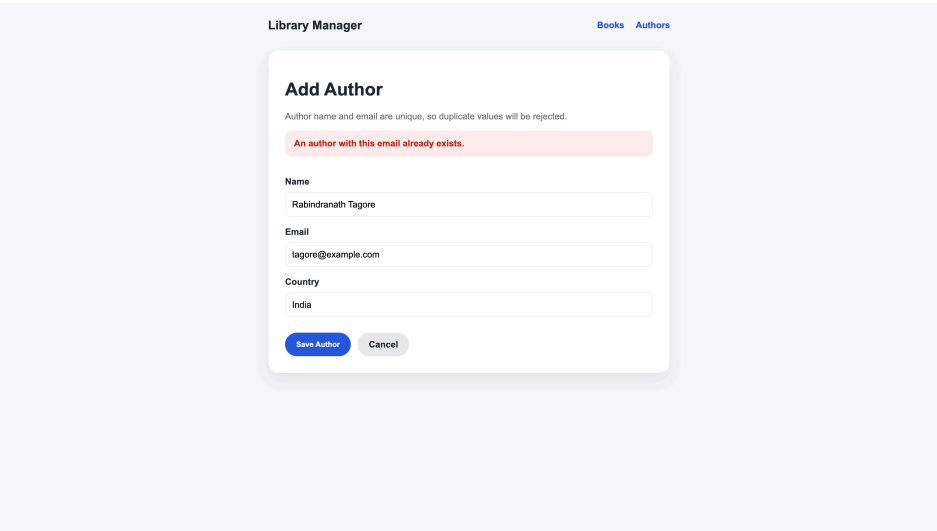

Figure 13: Duplicate author error handling.

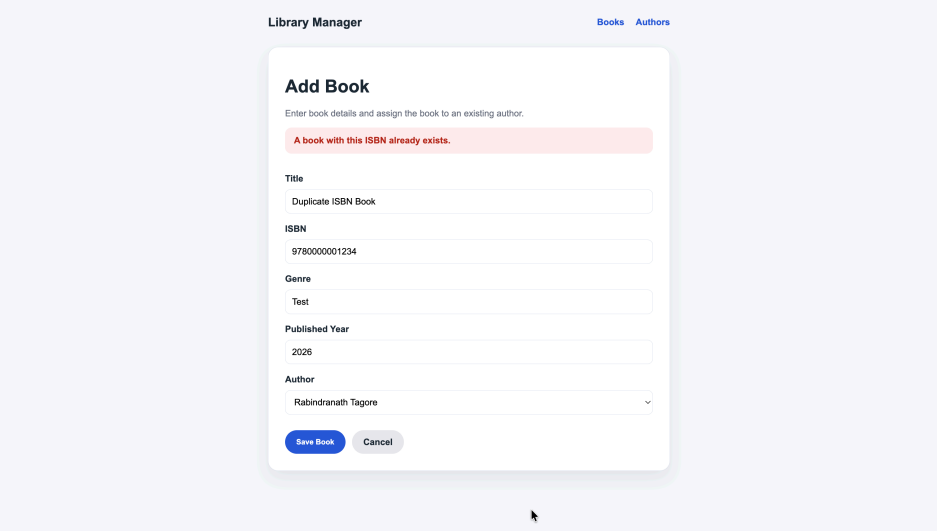

Figure 14: Duplicate book ISBN error handling.

12. H2 Database Verification

The H2 console was used to confirm table creation, sample data, created records, and the SQL inner join.

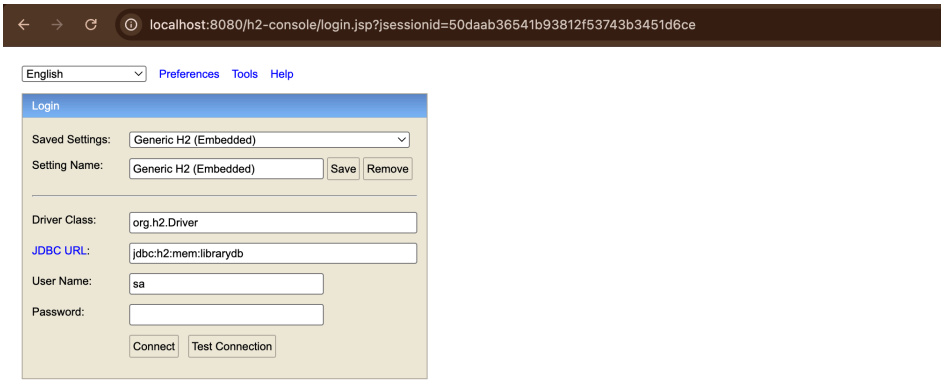

Figure 15: H2 console login.

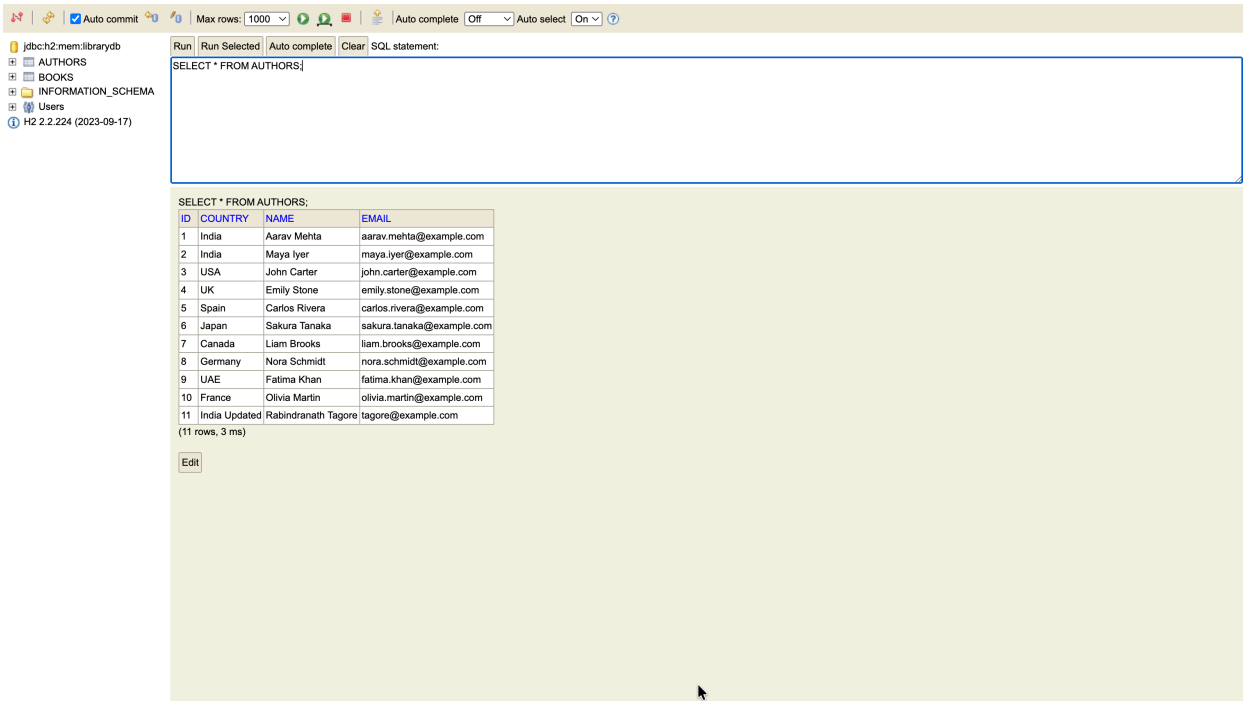

Figure 16: Authors table in H2.

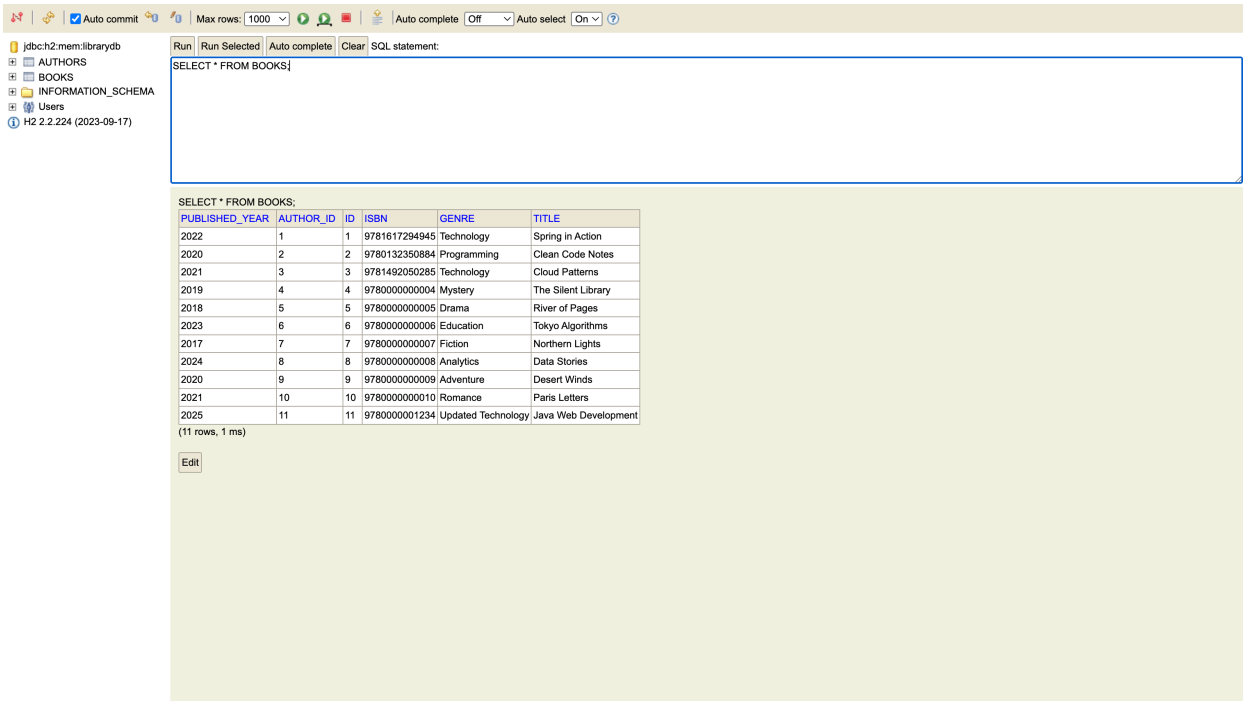

Figure 17: Books table in H2.

SQL join used for verification:

```
SELECT b.TITLE, b.ISBN, a.NAME AS AUTHOR_NAME, a.EMAIL
FROM BOOKS b
INNER JOIN AUTHORS a ON b.AUTHOR_ID = a.ID;
```

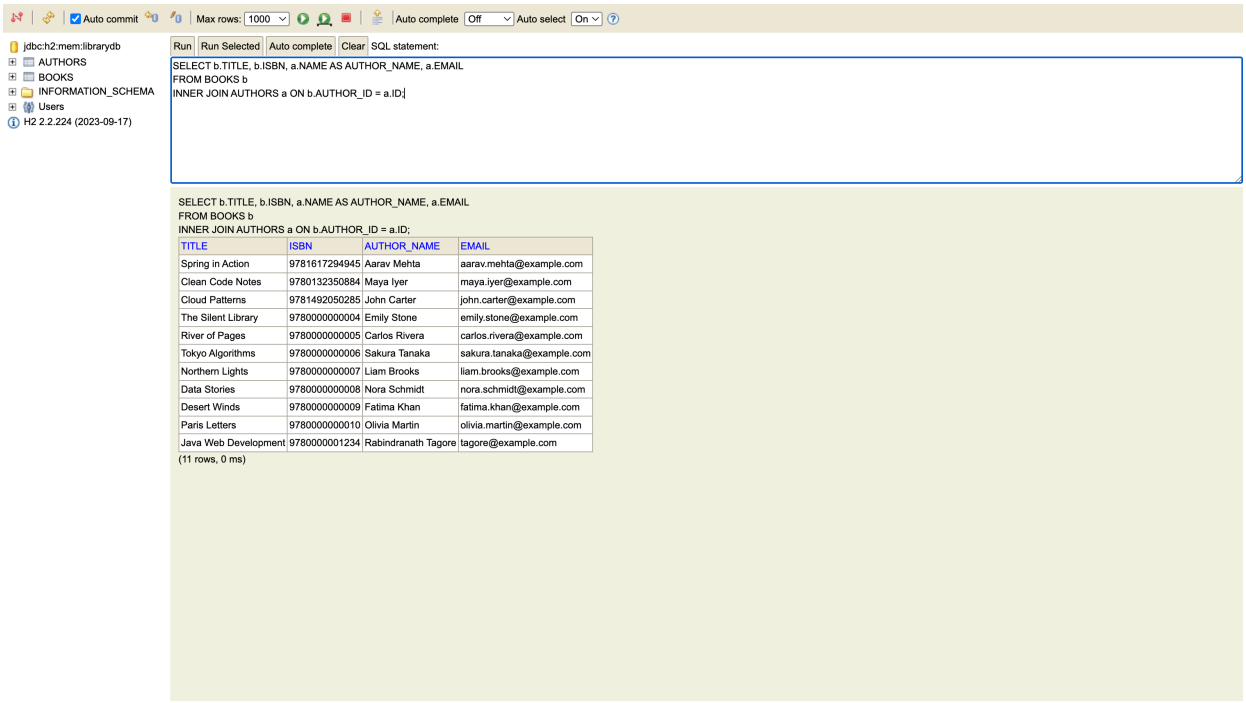

Figure 18: H2 inner join result.

13. Testing

Repository and service methods are tested using JUnit, Mockito, and Spring Boot test support.
Test coverage includes:

- Repository custom inner join method.
- Repository ISBN existence check.
- Service duplicate author email validation.
- Service duplicate book ISBN validation.
- Author update service method.
- Book update service method.

```
@DataJpaTest
class BookRepositoryTest {
    @Test
    void findBooksWithAuthorsReturnsInnerJoinRows() {
        Author author = authorRepository.save(
                new Author("Test Author", "test.author@example.com", "India"));
        bookRepository.save(
                new Book("Test Book", "9780000000999", "Education", 2024, author));

        List<BookAuthorView> rows = bookRepository.findBooksWithAuthors();

        assertThat(rows).hasSize(1);
        assertThat(rows.get(0).title()).isEqualTo("Test Book");
        assertThat(rows.get(0).authorName()).isEqualTo("Test Author");
    }
}
```

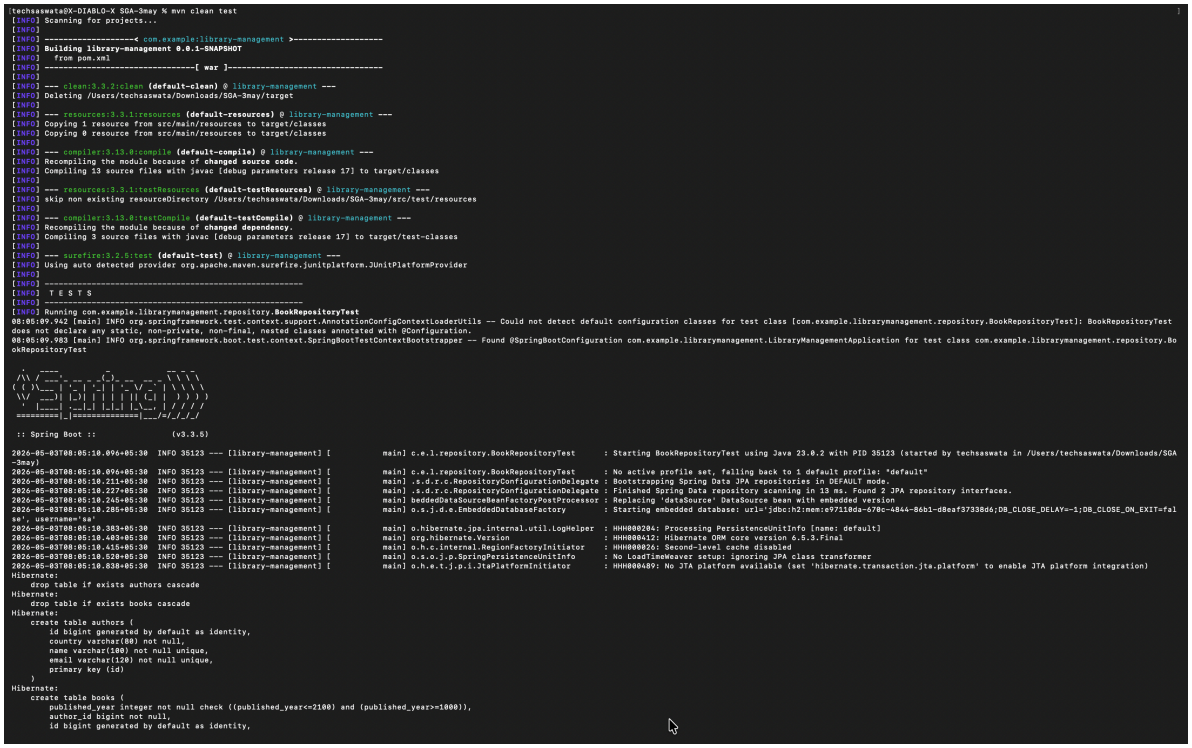

Figure 19: Maven test execution result.

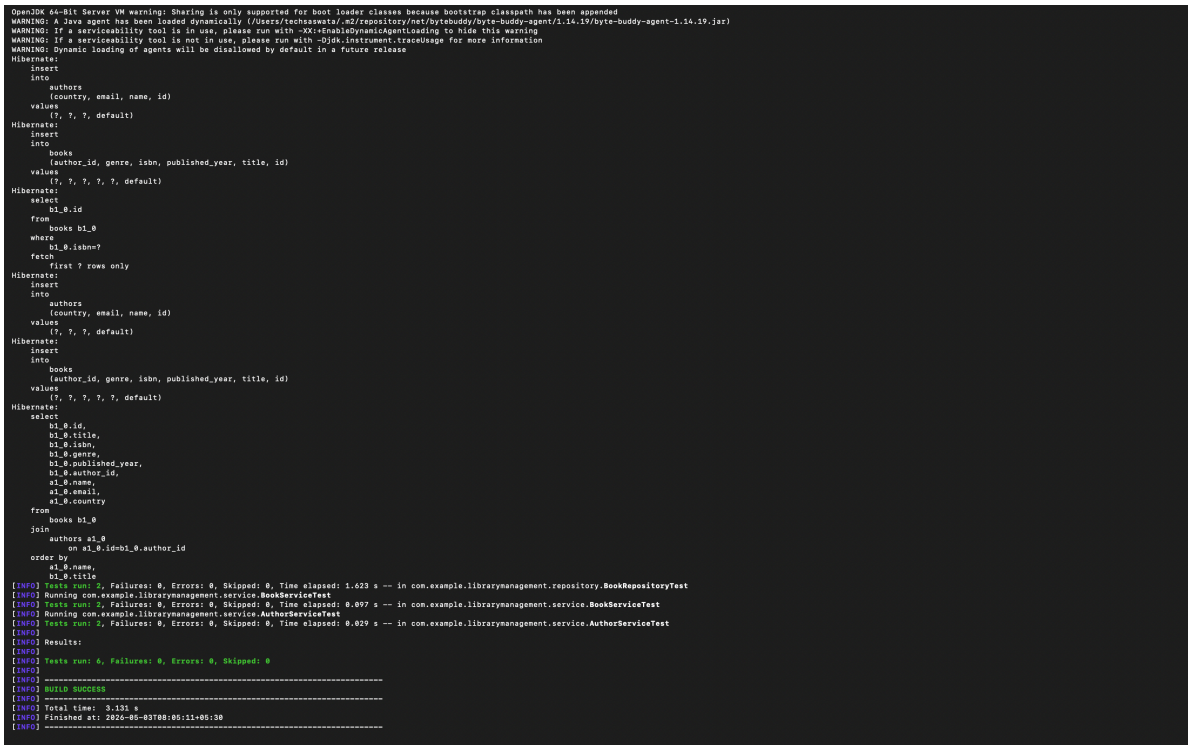

Figure 20: Maven test execution result continued.

14. Challenges Faced

1. Configuring JSP with Spring Boot 3 was not automatic because embedded Tomcat does not compile JSP pages unless the Jasper dependency is present, and JSTL now uses Jakarta namespaces. The issue was handled by adding `tomcat-embed-jasper`, Jakarta JSTL API, and the GlassFish JSTL runtime, then updating JSP taglib declarations to `jakarta.tags.core`.

2. The list page needed book and author data together. Rendering entities directly could create lazy-loading problems in JSP and would also expose more entity structure than required. This was handled with a repository-level JPQL inner join that returns a DTO projection, so the JSP receives only the fields needed for display.

3. The book form submits only the selected author's id, not a fully loaded author object. Saving that object directly can lead to invalid references or detached entity behavior. The service layer resolves the selected `Author` from the database first and then attaches it to the `Book` before saving.

4. Uniqueness had to be enforced at more than one level. Service checks give clearer feedback for common duplicate cases, while database unique constraints still protect the data if two requests try to insert the same email, name, or ISBN. Controllers catch `DataIntegrityViolationException` and return the user to the same form with an error message.

15. Conclusion

The project implements a complete Spring Boot MVC CRUD application for two related entities. It includes JPA table creation, sample data population, create/read/update operations, JSP views, CSS styling, repository and service layers, custom inner join query support, exception handling, and unit tests.

16. GitHub URL

GitHub URL: https://github.com/techSaswata/Library_Management_Assignment